Lab 15: Graphs

Task 1 : Implement a Graph Data Structure using a 2D Array in C++

Description:
In this lab task, you will implement a graph data structure using a 2D array. The program should allow the user to create a graph, add edges and vertices, and perform various operations on the graph, such as traversal and finding the shortest path.
Instructions:

1. Implement a class for the graph data structure using a 2D array representation.
2. The class should have functions for adding vertices, adding edges, and performing graph traversals.

.
Here are the necessary functions that the class should implement:

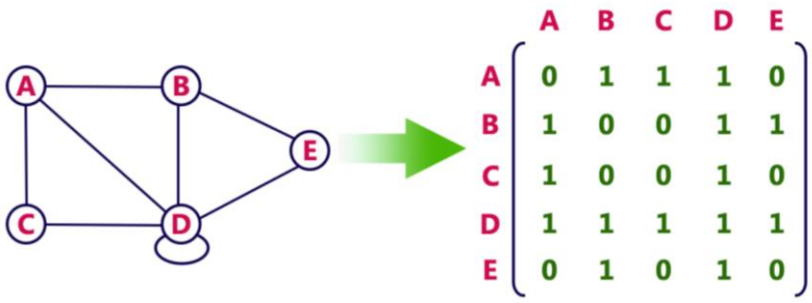
1. **Graph(int n)**: A constructor that creates an empty graph with n vertices.
2. **void addEdge(int from, int to, int weight)**: Adds an edge from vertex "from" to vertex "to" with a weight.
3. **void addVertex()**: Adds a new vertex to the graph.
4. **void printGraph():** Prints the graph.

```
class Graph {

    bool** adjMatrix;
    int numVertices;

    Graph(int numVertices);                 //constructor to set size of matrix
    void addEdge(int i, int j);             //marks the edge true in matrix
    void removeEdge(int i, int j);  //removes the edge at provided position
    bool isEdge(int i, int j);          //checks if edge exits or not, then returns
    void print();                       // prints the matrix

}
```



Task 2 : Implement a Graph Data Structure using a Node-based Approach (Adjacency list) in C++

In this lab, you will be implementing a graph data structure using a node-based implementation with an adjacency list. You will be using C++ for this lab.

To get started, create a **GraphNode** class that represents a node in the graph. This class should contain the following:

1. A data member that stores the ID of the node.
2. A data member that stores a list of the node's neighbors (i.e., nodes that are connected to this node).
3. A constructor that takes the ID of the node as an argument.
4. A method that adds a neighbor to the node.

Next, create a **Graph** class that represents the entire graph. This class should contain the following:

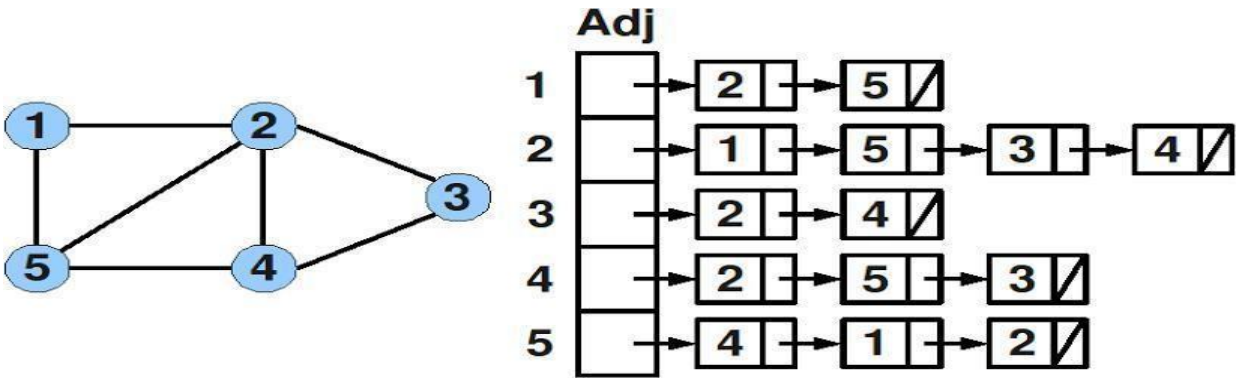
1. A data member that stores a list of all the nodes in the graph.
2. A method that adds a node to the graph.
3. A method that adds an edge between two nodes in the graph.
4. A method that prints out the adjacency list representation of the graph.
5. Your task is to implement the GraphNode and Graph classes as described above.

Data Items

Each vertex in a graph has a label (of type int) that uniquely identifies it. Vertices may include additional data.

Structure

The relationship between the vertices in a graph is expressed using a set of directed/undirected Edges, where each edge connects one pair of vertices. Here is an example of undirected graph and its representation in the form of adjacency list.



Operations

Node:

```
class Node
{
 public:
   int data;
   Node *
   next;
};
```

Adjacency List: // **it's a linked list class called adjlist**

```
class AdjList
{
 public:
   Node *head;
```

```
    Void Insert(Node * NewNode)
    Void Display()
};
```

Graph:

```
class Graph
{

public:
   int maxVertices;            //max no of vertices in your graph
   int noOfVertices;           // initially 0, increments with every call to createVertex
                               function. So     basically its kind of counter
   AdjList *list = NULL;
```

Graph (int maxVertices)

Results:

Constructor. Creates an empty graph. Allocates enough memory for a graph containing maxNumber vertices.

~ Graph ()

Results:

Destructor. Deallocates (frees) the memory used to store a graph.

Node * createVertex (int data)

Results:

Create and return a Vertex.

void insertEdge (int src, int dest)

Requirements:

The graph includes vertices source and destination.

Results:

Inserts an undirected edge connecting vertices source and destination into a graph.

void showGraphStructure ()

Results:

Outputs a graph with the vertices in array form and the edges in adjacency list form. If the graph is empty, outputs “Empty graph”. Note that this operation is intended for testing/debugging purposes only.

Task 3

Given an undirected graph, determine whether the graph is connected. A graph is considered connected if there is a path between every pair of vertices.

Write a function **is_connected(graph)** that takes an undirected graph represented as an adjacency list and returns True if the graph is connected, and False otherwise.

Example 1: Connected Graph

```
graph_connected = {
   1: [2],
   2: [1, 3, 4],
   3: [2],
   4: [2]
}
```

Example 2: Not Connected Graph

```
graph_not_connected = {
   1: [2],
   2: [1, 3],
   3: [2],
   4: [],
   5: []
}
```

Task - 4 (Bonus)

You are given a rectangular board of characters of size M x N and a dictionary of words. Your task is to find whether the given words can be formed by tracing a sequence of adjacent characters on the board. The word should not have multiple instances of the same cell, and you can move to any 8 adjacent characters in the board.

Write a function find_words(board, dictionary) that takes the board and dictionary as input and returns a list of all valid words found on the board.

Example:

```
board = [
    ['d', 'b', 'a', 'e'],
    ['w', 'f', 'c', 's'],
    ['a', 's', 'e', 'e']
]

# Example Dictionary
dictionary = ["see", "sees", "seesaw", "face", "bad", "badsee"]
```

The word “See”, “Sees” ”Seesaw” and “Face” exists in the board.